

PROJET DE LICENCE 3

CampusTasks

Gestion des tâches étudiantes

Flutter · Spring Boot · PostgreSQL · Docker

Version de travail 1.0.0

3 octobre 2026

Groupe de trois membres : à renseigner

Établissement et encadrant : à renseigner

Ce rapport présente le code et les procédures préparés localement. Les publications, le déploiement distant et les essais sur téléphone restent à réaliser. Ajouter les liens et captures après vérification réelle.

Sommaire

1. Contexte, objectifs et périmètre

Page 3

2. Architecture et conception des données

Page 4

3. Authentification et protection des comptes

Page 5

4. API REST, validation et règles métier

Page 6

5. Application Flutter et parcours utilisateur

Page 7

6. Docker, HTTPS et exploitation

Page 8

7. Versions, automatisation et distribution Android

Page 9

8. Vérifications, limites et préparation de la soutenance

Page 10

Lecture du rapport

Le document suit le parcours du besoin à la livraison : conception, sécurité, API, mobile, exploitation, versions et vérifications. Il décrit le code réalisé à partir du cahier des charges. Les conventions pourront être adaptées aux cours du professeur lorsqu'ils seront transmis.

1. Contexte, objectifs et périmètre

CampusTasks répond à un besoin courant : suivre plusieurs matières et organiser les devoirs, projets et révisions avec des échéances différentes. L'application rassemble ces activités dans une interface mobile. L'étudiant peut distinguer le travail à commencer, celui en cours et celui terminé, puis repérer les dates proches et les retards.

Le cahier des charges de licence 3 impose aussi une chaîne technique complète : application Flutter, API Java Spring Boot, base PostgreSQL, gestion des versions, Docker, déploiement HTTPS et distribution Android. Le résultat attendu ne se limite donc pas à des écrans. Il doit pouvoir être reproduit à partir du dépôt et expliqué par les trois membres du groupe.

Le périmètre fonctionnel retenu comprend la gestion des comptes, le CRUD des matières et des tâches, les filtres par matière et statut, le tri par date limite et le tableau de bord. Les notifications, le mode hors ligne et le partage de tâches restent hors de cette première livraison. Ce choix permet de concentrer les vérifications sur les parcours obligatoires.

Règles de gestion

Une matière appartient à un étudiant. Une tâche appartient à cet étudiant et référence une de ses matières. La suppression d'une matière est refusée tant qu'elle contient des tâches : l'utilisateur reçoit une explication et peut supprimer les tâches avant de recommencer. Une tâche terminée ne compte jamais comme retard, même si sa date limite est passée.

La date de création est générée par le serveur et conservée lors des modifications. La date limite est une date civile ; les indicateurs utilisent la date UTC, identique à Dakar. Les dates passées sont autorisées pour représenter du travail à rattraper. Le tableau de bord montre au maximum cinq échéances à venir et toutes les tâches en retard.

État du travail

Ce rapport décrit une réalisation locale issue de l'énoncé fourni. Les cours de l'enseignant ne sont pas encore disponibles ; leurs conventions ne sont pas supposées. Les noms des membres, captures et références de publication devront être ajoutés à partir des informations réelles du groupe.

2. Architecture et conception des données

Le flux principal est Flutter vers l'API REST, puis PostgreSQL. Le mobile envoie du JSON et reçoit des réponses structurées. En production, Caddy termine la connexion HTTPS et transmet les requêtes au backend sur le réseau Docker. PostgreSQL reste sur un réseau privé sans port publié.

Le backend comporte quatre couches. Le contrôleur associe les routes HTTP aux opérations. Les services appliquent les règles de gestion et délimitent les transactions. Les repositories Spring Data JPA effectuent les recherches et enregistrements. Les modèles représentent les tables. Les DTO définissent séparément les entrées et sorties du contrat HTTP ; les entités JPA ne sont pas directement exposées au client.

Table	Données principales
students	Identifiant, nom, email unique, mot de passe haché
subjects	Propriétaire, nom et description
tasks	Propriétaire, matière, titre, date limite, priorité, statut et création
auth_tokens	Propriétaire, empreinte du jeton, expiration

Un étudiant peut posséder plusieurs matières et plusieurs tâches. Une matière peut contenir plusieurs tâches. Les clés étrangères imposent l'existence des références. Les index sur le propriétaire et la date limite servent aux recherches les plus fréquentes. Les valeurs de priorité et de statut sont contraintes dans le SQL.

Le schéma est créé par une migration Flyway versionnée. Hibernate vérifie ensuite que les modèles correspondent à la base ; il ne modifie pas automatiquement le schéma en production. Toute évolution devra ajouter une nouvelle migration plutôt que modifier une migration déjà appliquée.

Responsabilité de l'isolation

Le service vérifie qu'une matière associée à une tâche appartient au même étudiant. Les repositories recherchent simultanément l'identifiant de l'objet et celui du propriétaire. Les clés étrangères seules ne suffisent pas à cette isolation : le contrôle applicatif reste indispensable. Le diagramme relationnel détaillé et le SQL initial sont fournis dans le dépôt.

3. Authentification et protection des comptes

L'inscription reçoit un nom, une adresse email et un mot de passe. L'email est normalisé en minuscules après suppression des espaces externes. Une contrainte d'unicité empêche deux comptes d'utiliser la même adresse. Le mot de passe est haché avec BCrypt ; le texte original n'est pas conservé dans PostgreSQL.

La connexion compare le mot de passe saisi au hachage. En cas d'échec, la réponse ne distingue pas compte absent et mauvais mot de passe. Une connexion réussie produit un jeton opaque aléatoire de 256 bits. Le serveur conserve uniquement son empreinte SHA-256, l'étudiant associé et sa date d'expiration. Par défaut, la session est valable vingt-quatre heures.

Flutter stocke le jeton dans le stockage sécurisé du système, puis le transmet dans l'en-tête Authorization avec le préfixe Bearer. Le filtre Spring Security vérifie son empreinte et son expiration avant de définir l'identité de la requête. Le client ne fournit pas l'identifiant de propriétaire qui fait autorité : il est extrait de l'authentification.

Déconnexion et expiration

La déconnexion supprime la session correspondante en base et efface le jeton du téléphone. Une réutilisation de ce jeton est refusée. En l'absence de réseau, le mobile efface néanmoins sa session locale ; le jeton distant conserve sa validité jusqu'à expiration s'il n'a pas pu être révoqué. Cette limite est documentée.

Le choix d'un jeton opaque facilite la révocation immédiate et évite une clé de signature JWT. En contrepartie, l'API interroge la base pour authentifier chaque requête. Ce compromis convient au périmètre du projet. Une purge des sessions expirées et une limitation des tentatives de connexion pourront être ajoutées pour une exploitation prolongée.

Accès aux objets

Les routes métier sont protégées. Un objet absent ou appartenant à un autre étudiant donne une réponse 404, afin de ne pas révéler son existence. Les tests essaient de modifier, supprimer et associer des éléments d'un autre compte. Les DTO de réponse ne contiennent ni hachage de mot de passe ni empreinte de session.

4. API REST, validation et règles métier

Le contrat est préfixé par /api/v1. Cette version de contrat reste stable pour les évolutions compatibles, indépendamment de la version du logiciel. Les routes comptes permettent inscription, connexion et déconnexion. Les routes matières et tâches offrent création, liste, modification et suppression. Le dashboard regroupe les indicateurs.

Les méthodes HTTP expriment l'opération : GET lit, POST crée ou ouvre une session, PUT modifie un objet complet et DELETE supprime. Les créations retournent 201, les lectures et modifications 200, les suppressions réussies 204. Une erreur de saisie retourne 400 ; une session absente ou invalide retourne 401 ; un objet inaccessible retourne 404 ; une matière non vide ou un conflit d'unicité retourne 409.

Les DTO utilisent les contraintes Jakarta Validation. Le titre est obligatoire et limité à 150 caractères, le nom de matière à 100. Les descriptions peuvent être vides mais sont limitées en longueur. La date, le statut, la priorité et la matière sont obligatoires pour une tâche. Les valeurs possibles sont explicitement définies par des énumérations.

Filtres et tableau de bord

La liste de tâches accepte une matière, un statut et un sens de tri. Les filtres peuvent être combinés. Le tri utilise la date limite puis l'identifiant pour obtenir un ordre déterministe. Filtrer sur une matière étrangère est refusé, plutôt que retourner une liste ambiguë.

Les compteurs distinguent TODO, IN_PROGRESS et DONE. Les échéances à venir excluent les tâches terminées et les dates déjà dépassées. Les retards correspondent à une date strictement antérieure à aujourd'hui et un statut différent de DONE. Le passage à DONE retire donc immédiatement une tâche de cette liste.

Erreurs cohérentes

Le gestionnaire d'erreurs renvoie un objet JSON portant un message compréhensible. Les formats de date ou d'énumération invalides sont traités. Les violations d'intégrité sont traduites en conflit, notamment si deux opérations concurrentes rencontrent une référence encore utilisée. La documentation API et la collection api.http permettent de rejouer les requêtes sans passer par Flutter.

5. Application Flutter et parcours utilisateur

L'application comporte un écran de connexion et d'inscription, un tableau de bord, une liste de tâches, une liste de matières et un formulaire réutilisé en création et modification. La navigation principale distingue accueil, tâches et matières. Chaque élément peut être modifié en ouvrant sa fiche ; les suppressions demandent une confirmation.

Le client API centralise l'adresse du backend, les en-têtes JSON et Bearer, les délais d'attente et l'interprétation des erreurs. L'adresse est fournie à la compilation par `API_BASE_URL`. Le développement sur émulateur utilise l'adresse spéciale 10.0.2.2 pour joindre le PC. La livraison doit utiliser l'URL HTTPS du serveur autorisé.

Gestion de session et états

Au démarrage, SessionGate lit le stockage sécurisé. Sans jeton, l'écran de connexion apparaît. Avec une session, les données métier sont chargées. Une réponse 401 efface le jeton et ramène à l'authentification. L'application affiche un indicateur pendant le chargement, un message et une action de réessai en cas d'échec, et un état explicite pour une liste vide.

Les formulaires vérifient les champs obligatoires avant l'envoi et affichent aussi les erreurs de l'API. Le bouton d'enregistrement est désactivé pendant la requête pour limiter les soumissions répétées. Une tâche requiert une matière ; si aucune matière n'existe, l'application invite l'étudiant à en créer une.

La liste de tâches expose les filtres de matière et de statut ainsi que le sens du tri. Le dashboard montre les compteurs et les échéances. Les tâches en retard reçoivent une indication visuelle et textuelle. L'étudiant peut modifier le statut et constater l'évolution des indicateurs après rechargement.

Choix et limites

La gestion d'état utilise les mécanismes Flutter de base afin de garder le projet lisible. Elle pourra être adaptée à la méthode enseignée après réception des cours. Les données ne sont pas disponibles hors ligne ; le backend reste la source de vérité. L'analyse statique et les tests locaux ne remplacent pas une vérification sur téléphone, notamment pour le stockage sécurisé et le réseau.

6. Docker, HTTPS et exploitation

Le Dockerfile sépare compilation et exécution. L'étape Maven construit le JAR avec Java 21. L'image finale contient un environnement Java d'exécution et le JAR, puis lance l'application avec un utilisateur sans privilèges administrateur. Les paramètres de connexion à la base et de durée de session sont reçus par variables d'environnement.

Compose définit PostgreSQL et l'API. Le healthcheck PostgreSQL contrôle sa disponibilité avant le démarrage de l'API. Un volume nommé conserve les données lorsque les conteneurs sont remplacés. Les services redémarrent sauf arrêt explicite. Les journaux API sont bornés afin de limiter leur croissance.

La configuration de développement publie l'API uniquement sur l'interface locale du PC. La configuration de production ajoute Caddy sur les ports 80 et 443. Le proxy obtient et renouvelle un certificat pour un domaine réel. PostgreSQL n'a aucun port publié et demeure sur le réseau privé. Le fichier exemple contient des valeurs fictives ; la configuration réelle est exclue de Git.

Mise à jour et retour arrière

Le déploiement utilise une image portant un tag précis. Avant une mise à jour, l'opérateur sauvegarde la base et note la version en cours. Il change la version de l'image, la récupère et recrée l'API, puis vérifie les parcours et les journaux. Un endpoint de vivacité indique que Java répond ; une requête métier authentifiée est nécessaire pour contrôler également PostgreSQL.

Revenir à une ancienne image n'annule pas une migration de base. Si le schéma a changé de façon incompatible, il faut une migration corrective ou une restauration contrôlée. Restaurer une sauvegarde peut perdre les écritures postérieures : cette décision ne doit pas être implicite.

Sauvegarde

La procédure utilise `pg_dump` au format custom, copie le fichier hors du conteneur et recommande une sauvegarde chiffrée hors du serveur. La restauration doit être répétée sur un environnement isolé. Les commandes sont préparées dans la documentation ; aucune exécution Docker ou publication distante n'est présentée comme vérifiée à ce stade.

7. Versions, automatisation et distribution Android

Le produit vise une version commune 1.0.0 : tag Git v1.0.0, backend 1.0.0, image Docker 1.0.0 et Flutter 1.0.0+1. Le suffixe Android représente le numéro de build et doit augmenter pour une mise à jour. Les tags et notes de release permettront de retrouver la correspondance entre code, image et APK.

Le workflow de contrôle sur pull request compile et teste le backend, vérifie le format Dart, lance l'analyse Flutter et ses tests. Le workflow de tag contrôle la cohérence des versions, exécute les tests backend, construit l'image et la publie sur Docker Hub avec le tag du produit. Les identifiants nécessaires sont fournis par les secrets GitHub, jamais dans le code.

Organisation à trois

La branche main contient les versions stables. Les fonctionnalités se développent sur des branches dédiées et sont intégrées par une pull request relue. Une répartition API, mobile et déploiement est proposée, avec relecture croisée. Les noms, contributions et liens de PR devront refléter le travail réellement effectué, sans reconstruction artificielle d'un historique collectif.

Signature et APK

La livraison Android utilise une clé privée créée et conservée hors du dépôt. Le fichier local `key.properties` configure Gradle ; une construction release refuse l'absence de cette configuration. Le projet ne signe pas silencieusement une livraison avec une clé debug.

La compilation release reçoit l'URL HTTPS et les numéros de version et de build. Après construction, la signature et l'empreinte du fichier sont vérifiées. L'APK est installé sur un appareil, puis les parcours sont rejoués contre le backend distant. Une seconde version doit utiliser la même clé et le même `applicationId`, avec un numéro de build supérieur, pour mettre à jour l'installation existante.

La GitHub Release doit contenir l'APK, son empreinte et une notice. La clé de signature n'y figure jamais. Le build debug sert uniquement à la vérification technique locale ; il ne prouve ni la signature de livraison ni la communication avec un serveur distant.

8. Vérifications, limites et préparation de la soutenance

Les tests d'intégration backend emploient les couches réelles de contrôleur, sécurité, service et JPA, avec H2 en mode PostgreSQL. Trois scénarios ont passé sans échec : règles métier et isolation, validation/authentification/révocation, reconnexion/filtres/tri. Ils contrôlent notamment les mutations d'objets étrangers, l'association à une matière étrangère et le refus de suppression d'une matière non vide.

Un test vérifie que le mot de passe stocké diffère du texte original. La déconnexion est suivie d'une requête avec l'ancien jeton, qui reçoit 401. Le scénario de reconnexion retrouve les tâches après ouverture d'une nouvelle session. Le tableau de bord exclut un retard passé à DONE. L'analyse Flutter ne signale aucun problème. Les cinq tests mobiles couvrent l'écran d'inscription, la restauration du jeton, son effacement sur une réponse 401, la déconnexion malgré une panne serveur et la lecture des réponses JSON accentuées ou vides.

Portée des résultats

Ces résultats sont locaux. H2 ne prouve pas le comportement de PostgreSQL en exploitation. Les essais de persistance après redémarrage Docker, de sauvegarde/restauration, d'HTTPS distant et de mise à jour signée sur téléphone restent nécessaires. Le registre de vérifications du dépôt distingue les résultats exécutés des procédures à réaliser. Les captures et logs devront être datés et débarrassés des secrets.

Les limites fonctionnelles sont assumées : aucune notification, récupération de mot de passe, collaboration ou synchronisation hors ligne. Les listes ne sont pas paginées. Pour une exploitation prolongée, il faudra ajouter purge des sessions expirées, limitation des tentatives login et monitoring adapté à la plateforme.

Démonstration de quinze minutes

Le scénario prévoit deux minutes de contexte et architecture, puis compte/matière, CRUD et filtres de tâches, dashboard, reconnexion et isolation. Les dernières minutes couvrent Docker, HTTPS, CI, image versionnée, signature APK et limites. Les trois membres doivent pouvoir expliquer l'ensemble du flux, même si leurs responsabilités diffèrent.

Éléments à finaliser

Renseigner les membres du groupe, adapter les conventions aux cours reçus, valider PostgreSQL et Docker, publier sur les comptes autorisés, déployer sur le serveur retenu et tester la livraison Android. Remplacer ensuite les mentions provisoires par des résultats et liens vérifiables. Le cahier des charges fourni constitue la référence du périmètre ; les fichiers du dépôt décrivent les choix de réalisation.